Unit-3
API Integration and Tools

By: Dr. A. R. Welekar

OpenAI API

- The OpenAI API enables developers to integrate advanced AI models into applications with ease.

Three foundational components of OpenAI API

1. Authentication – to securely access the API.

2. Completion – for text-based prompt-response generation.

3. Chat endpoint – for conversational AI experiences using messages.

Authentication in OpenAI API

1.1 What is Authentication?

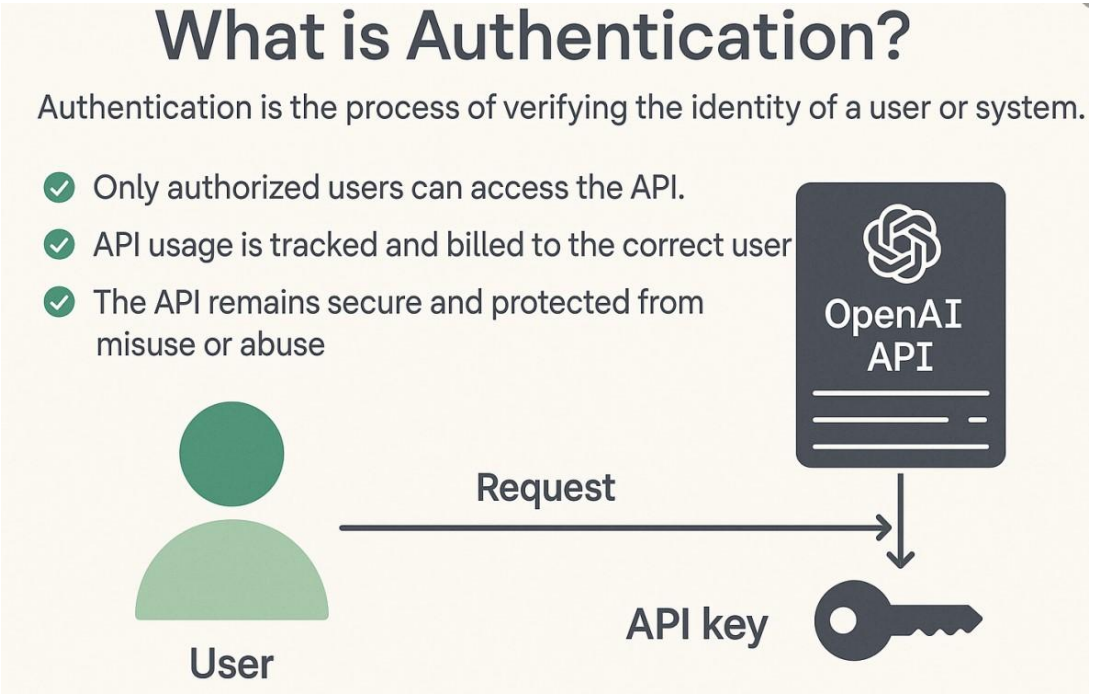
Authentication is the process of verifying the identity of a user or system. In the context of OpenAI API, authentication ensures:

- Only authorized users access the API.
- Usage is monitored and billed correctly.
- The API remains secure from abuse.

Authentication in OpenAI API

1.2 API Key

- When you register at platform.openai.com, OpenAI provides a secret API key. This key must be included in the HTTP Authorization header of every API request.



Authentication in OpenAI API

1.3 How to Authenticate

Header Format (HTTP):

Authorization: Bearer sk-XXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXX

Python Example:

```
import openai
openai.api_key = "sk-XXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXX"          # Set your API key
# Now you can make API calls
response = openai.Completion.create(
  engine="text-davinci-003",
  prompt="Hello, world!",
  max_tokens=5
)
print(response.choices[0].text)
```

Completion in OpenAI API

2. The Completion Endpoint

The Completion endpoint is designed to return AI-generated text based on an input prompt. It is ideal for one-shot tasks where you provide a single prompt and expect a response.

- Example Use Cases: Story writing, code generation, translation, product descriptions.

Completion in OpenAI API

2.1 Endpoint Structure

Endpoint URL:

POST https://api.openai.com/v1/completions

2.2 Key Parameters:

Parameter	Description
model	Model to use (e.g., text-davinci-003)
prompt	The input text
max_tokens	Maximum number of tokens to generate
temperature	Creativity control (0 = deterministic, 1 = random)
stop	Optional stop sequences

Completion in OpenAI API

The "model" parameter in the Completion endpoint of the OpenAI API specifies which language model you want to use for generating the completion.

◈ Purpose of the "stop" Parameter:
It tells the API which trained model should be used to process your request, as different models have different capabilities, costs, and speeds.

◈ Common Models for Completion Endpoint:

Model	Description
text-davinci-003	Most powerful and accurate model for text completion (GPT-3.5 generation)
text-curie-001	Less powerful than davinci, but faster and cheaper
text-babbage-001	Faster, cheaper, and lower-quality
text-ada-001	Fastest and cheapest, suitable for simple tasks

Completion in OpenAI API

◈ Model Selection Tips:

- Use text-davinci-003 for high-quality, complex completions.
- Use text-curie-001 for summarization or classification.
- Use text-ada-001 for lightweight tasks (e.g., simple data formatting).

```
openai.ChatCompletion.create(
  model="gpt-4",
  messages=[{"role": "user", "content": "What's the capital of
Italy?"}]
)
```

Completion in OpenAI API

The "stop" parameter in the OpenAI Completion API is used to control where the text generation should stop. It tells the model to end its response when it encounters one of the specified stop sequences.

◈ Purpose of the "stop" Parameter:
To prevent the model from generating unwanted or excessively long text by defining custom end conditions.

◈ Syntax (in Python): Example-1(Q&A Format)

```
response = openai.Completion.create(
  engine="text-davinci-003",
  prompt="Q: What is the capital of France?\nA:",
  max_tokens=50,
  stop=["\n", "Q:"]
)
```

◈ Syntax (in Python): Example-2 (Code Generation)

```
response = openai.Completion.create(
  engine="text-davinci-003",
  prompt = "def add(a, b):",
  max_tokens=50,
  stop = ["# End"]
)
```

Completion in OpenAI API

2.3 Example: Generating Text

```
response = openai.Completion.create(
    model="text-davinci-003",
    prompt="Describe the structure of a plant cell.",
    max_tokens=100,
    temperature=0.5
)
print(response.choices[0].text.strip())
```

Key Parameters:

Completion in OpenAI API

2.4 Output

The response returns a JSON object containing generated text.

Example:


```
{
  "choices": [
    {
      "text": "A plant cell consists of a cell wall, cell membrane, nucleus...",
      "finish_reason": "stop"
    }
  ]
}
```

Chat Endpoint in OpenAI API

3. The Chat Endpoint

The Chat endpoint is used for creating conversational agents. Instead of a single prompt, it takes a list of message objects that simulate a conversation. This is how ChatGPT itself works.

3.1 Endpoint Structure

Endpoint URL: POST https://api.openai.com/v1/chat/completions

Chat Endpoint in OpenAI API

3.2 Roles in Messages

Each message has a role:

Role	Description
system	Sets the assistant’s behaviour or persona
user	Represents user input
assistant	Represents the assistant's prior response

Chat Endpoint in OpenAI API

3.3 Example: Chat Completion

```
response = openai.ChatCompletion.create(
    model="gpt-4",
    messages=[
        {"role": "system", "content": "You are a helpful academic tutor."},
        {"role": "user", "content": "Can you explain the Pythagorean theorem?"}
    ]
)
print(response.choices[0].message["content"])
```

Chat Endpoint in OpenAI API

3.4 Output


```
{
  "choices": [
    {
      "message": {
        "role": "assistant",
        "content": "Certainly! The Pythagorean theorem states that in a right-angled triangle..."
      }
    }
  ]
}
```

Chat Endpoint in OpenAI API

3.5 Benefits

- Maintains multi-turn conversations.
- Allows for consistent tone and personality using system prompts.
- Supports function calling and streaming (advanced features).

Chat Endpoint in OpenAI API

3.6 Comparison Table

Feature	Completion Endpoint	Chat Endpoint
Input	Single prompt string	List of messages (role, content)
Context Handling	Limited (one-shot)	Built-in memory via message history
Model	text-davinci-003, etc.	gpt-3.5-turbo, gpt-4
Best For	Text generation, code, essays	Conversational apps, chatbots
Personalization	Manual through prompt	Easy via system message

Hugging Face Transformers API and Hosted Models

Hugging Face is an AI company that provides open-source tools and APIs to work with state-of-the-art natural language processing (NLP) models such as BERT, GPT, RoBERTa, T5, and many others. Their most popular offering is the Transformers library, which simplifies using pre-trained transformer models.

Hugging Face Transformers API and Hosted Models

4.1 Transformers API:

- The Transformers API is a Python library provided by Hugging Face that lets you easily:
- Load and use pretrained NLP models (e.g., BERT, GPT-2, T5)
- Fine-tune models on custom data
- Tokenize input text
- Generate, classify, or translate text
- Use models across tasks: classification, generation, QA, summarization, etc.

Hugging Face Transformers API and Hosted Models

4.1.1 Core Components of the Transformers API:

1. Models:
   - Classes like BertModel, GPT2LMHeadModel, T5ForConditionalGeneration.
   - Pretrained models are accessed via from_pretrained().
2. Tokenizers:
   - Convert raw text into input IDs suitable for transformer models.
   - Handles padding, truncation, special tokens.
3. Pipelines:
   - High-level interface for common NLP tasks.
   - Example: sentiment-analysis, text-generation, translation, etc.
4. Datasets and Trainer:
   - For training/fine-tuning models using PyTorch or TensorFlow.

Hugging Face Transformers API and Hosted Models

4.1.2 Hosted Models on Hugging Face Hub:

Hugging Face hosts thousands of pre-trained models on their platform: https://huggingface.co/models

1. Each model is:
   - Version-controlled
   - Documented
   - Usable directly via the Transformers library
2. You can:
   - Search models by task (text classification, summarization, etc.)
   - Run them via the hosted inference API
   - Download them to use locally

Hugging Face Transformers API and Hosted Models

4.1.3 Example: Using GPT-2 with Hugging Face Transformers:

1. Installation

pip install transformers

This uses a hosted pre-trained GPT-2 model downloaded locally via the API.

2. Load GPT-2 and Generate Text

```
from transformers import GPT2LMHeadModel, GPT2Tokenizer

    # Load tokenizer and model
    tokenizer = GPT2Tokenizer.from_pretrained("gpt2")
    model = GPT2LMHeadModel.from_pretrained("gpt2")

    # Encode input prompt
    input_ids = tokenizer.encode("Once upon a time", return_tensors='pt')

    # Generate output
    output = model.generate(input_ids, max_length=50, num_return_sequences=1)

    # Decode and print result
    print(tokenizer.decode(output[0], skip_special_tokens=True))
```

Hugging Face Transformers API and Hosted Models

4.1.3 Example: Use Inference API for Hosted Models:

You can call models hosted on Hugging Face servers via a simple HTTP API or Python.

1. Inference via Python

```
from transformers import pipeline


# Load hosted sentiment analysis pipeline
classifier = pipeline("sentiment-analysis")


# Run prediction
result = classifier("I love Hugging Face Transformers!")
print(result)
```

Hugging Face Transformers API and Hosted Models

- Hugging Face Transformers API and Hosted Models provide a powerful, flexible, and developer-friendly way to work with the latest AI models without the burden of training from scratch.

- Whether you're doing zero-shot inference, fine-tuning, or prompt-based engineering, the ecosystem supports research, prototyping, and production.

LangChain Introduction

❖LangChain Introduction:

- LangChain is a powerful framework that enables developers to build applications powered by language models (LLMs) by chaining together multiple components like prompts, models, tools, and memory in a modular and composable way.

- LangChain is especially useful for building multi-step reasoning tasks, agent-based applications, question answering systems, chatbots, and LLM-powered workflows.

LangChain

1.Agents in LangChain:

➢ Definition:

An Agent in LangChain is an intelligent controller that decides what actions to take based on the input and available tools. Unlike static chains, agents use reasoning and dynamic decision-making to determine what steps to perform next.

➢ How it works:

- An agent receives a task (e.g., "What's the weather in Mumbai and send a summary email to me").
- It evaluates what tools are available (e.g., a weather API, email service).
- It selects tools dynamically, executes intermediate steps, observes the results, and continues until the task is completed.

LangChain

1.Agents in LangChain:

➢Common Agent Types:

- ReactAgent (Reasoning and Acting)
- ChatAgent (used for multi-turn conversation)
- Plan-and-Execute Agent (splits planning and tool usage)

LangChain

2.Chains in LangChain:

A Chain is a sequence of calls or components that connects LLMs with other elements like prompts, inputs/outputs, or tools. It represents a static flow of how an input gets processed.

➢ **Example:**

- Simple Chain: User Input → Prompt Template → LLM → Output Parser → Final Output
- Multi-step Chain: Chain A → Chain B → Chain C

➢ **Built-in Chains:**

- LLMChain: Most basic, combines a prompt + LLM.
- SequentialChain: Runs multiple chains in sequence.
- SimpleSequentialChain: Passes outputs from one chain directly to the next.
- RouterChain: Dynamically routes input to different chains.

LangChain

3.Tools in LangChain:

A Tool is an external function or service that an agent can call to complete a task. Tools allow LLMs to go beyond text prediction and interact with the world (e.g., APIs, databases, Python code).

- **Typical Tools:**
  - Web search (SerpAPI)
  - Calculator
  - Python REPL
  - SQL Database Query
  - File reader/writer
  - Custom APIs (e.g., weather, stock, email)

Vector Database

- When you work with Large Language Models (LLMs) or any AI model, the model itself doesn't "search" the internet or your database in real time. Instead, you give it relevant context in the prompt.
- But if you have millions of documents, how do you quickly find the most relevant ones to pass into the LLM?
- That's where vector databases like FAISS and ChromaDB come in.
- Modern AI systems often need to search and retrieve the most relevant information from a huge dataset — not as exact keyword matches, but based on semantic similarity.
- Instead of storing just text, they:
- Convert the text into vectors (embeddings) using a model (e.g., OpenAI, BERT, SentenceTransformers).
- Store those vectors in a database optimized for similarity search.
- Search by finding vectors close to the query vector (nearest neighbors).

FAISS (Facebook AI Similarity Search)

FAISS is an open-source library by Facebook AI for fast nearest neighbor search in large collections of high-dimensional vectors.

How it Works:

- Create an index and insert all vector embeddings.
- When a query comes in, convert it into a vector.
- FAISS finds the closest vectors (k-nearest neighbors).
- Return the related items/documents.

When to use FAISS:

- When speed is critical.
- When your data is already preprocessed as embeddings.
- In Retrieval-Augmented Generation (RAG) pipelines.
- For semantic search, recommendation systems, image similarity, etc.

FAISS (Facebook AI Similarity Search)

Key Features:

- High-performance similarity search (cosine similarity, L2 distance).
- Scales to millions/billions of vectors.
- Supports GPU acceleration for speed.
- Works offline (in-memory or disk-based index).

ChromaDB (Facebook AI Similarity Search)

ChromaDB is an open-source vector database designed for LLM applications (especially Retrieval-Augmented Generation).

Unlike FAISS (just an indexing library), ChromaDB is a full database system with:

- Storage
- Metadata management
- Filtering
- Persistence
- Integration with AI toolchains

Features:

- Stores both embeddings + metadata (like document IDs, text, source).
- Built-in persistence — data is stored to disk automatically.
- Filtering — search within specific metadata filters.
- Embeddings integration — can generate embeddings on the fly using models.
- LangChain support — widely used for AI applications.

JSON Output Formatting

- JSON (JavaScript Object Notation) is a lightweight text format for representing structured data.
- When we say “output in JSON format,” we mean structuring the response like:

{

"name": "Amit W.",

"role": "Deputy Dean of Academics",

"interests": ["AI", "IoT", "Cybersecurity"],

"active_projects": 3

}

JSON Output Formatting

Why use JSON for output?

- Machine-readable → Easy for programs to parse and process.
- Language-agnostic → Works with Python, JavaScript, Java, C#, etc.
- Standardized → Avoids ambiguity in communication between systems.

When two systems (or a human and a system) need to exchange data, the sender formats the information as JSON so the receiver can:

- Easily parse it (machine-friendly)
- Understand the structure (predictable keys and values)

JSON Output Formatting

How It's Created?

JSON is just text that follows specific syntax rules:

- Key–Value pairs inside { }
- Keys are always strings in double quotes (" ")
- Values can be:
  - String ("Hello")
  - Number (42)
  - Boolean (true or false)
  - Null (null)
  - Array ([])
  - Another JSON object ({})
- Commas separate pairs, but no trailing commas allowed

JSON Output Formatting

Example creation in Python:

```
import json
data = {
    "name": "Amit",
    "role": "Deputy Dean",
    "projects": ["AI", "IoT", "Cybersecurity"],
    "active": True
}
json_output = json.dumps(data)  # Convert to JSON string
print(json_output)
```

Output:

```
{"name": "Amit", "role": "Deputy Dean", "projects": ["AI", "IoT", "Cybersecurity"], "active": true}
```

JSON Output Formatting

How It's Sent?

The JSON text can be:

- Returned by an API (HTTP response)
- Written into a .json file
- Embedded in chatbot output
- Passed between frontend and backend in a web app

Example API response (HTTP):

HTTP/1.1 200 OK

Content-Type: application/json

{"temperature": 29.4, "unit": "Celsius"}

JSON Output Formatting

How It's Parsed?

Once received, the JSON string is decoded into a native data structure in the target programming language.

Python parsing:

```
parsed_data = json.loads(json_output)

print(parsed_data["projects"][0])  # Output: AI
```

JavaScript parsing:

```
let parsedData = JSON.parse(json_output);

console.log(parsedData.projects[0]); // Output: AI
```

JSON Output Formatting

Real-Life Example in Action

Scenario: You query a chatbot for weather data.

Prompt:"Give me today's Mumbai weather as JSON with keys: temperature, condition, humidity"

Bot Output (JSON formatted):

```
{
  "temperature": 29.4,
  "condition": "Partly Cloudy",
  "humidity": 76
}
```

Your App:

- Parses the JSON
- Displays it as:

  🌤29.4°C | Partly Cloudy | Humidity: 76%